CS 5542 — Big Data and AI Technologies Project Assignment 2 — Phase 2

End-to-End Multimodal RAG Pipeline

Behavioral Fingerprinting of Multi-Agent AI Attack Swarms

Student	Murali Krishna Goud Ediga
Course	CS 5542 — Big Data and AI Technologies
Semester	Spring 2026
Submission	Project Assignment 2 (Phase 2)
Due Date	February 28, 2026, 11:59 PM CST
GitHub	https://github.com/muralikrish9/CS5542
Demo Video	https://youtu.be/ZInB_jDfea0
Snowflake	https://app.snowflake.com/streamlit/sfedu02/dcb73175/#/apps/wjwddq3xfqomayzpws35

This report documents the Phase 2 consolidation of Labs 1–5 into a reproducible, end-to-end Retrieval-Augmented Generation (RAG) pipeline for cybersecurity research. The domain focuses on **behavioral fingerprinting of multi-agent AI attack swarms** — a research area at the intersection of adversarial machine learning, network security, and multi-agent systems. The pipeline spans text and multimodal data ingestion, advanced retrieval with reranking, Streamlit application deployment, and Snowflake-based analytical data warehousing.

Pipeline Architecture Diagram

The diagram below illustrates the complete end-to-end data flow from raw data sources through the multimodal knowledge base, retrieval pipeline, Snowflake warehouse, and application interface. Each node maps to the corresponding lab.

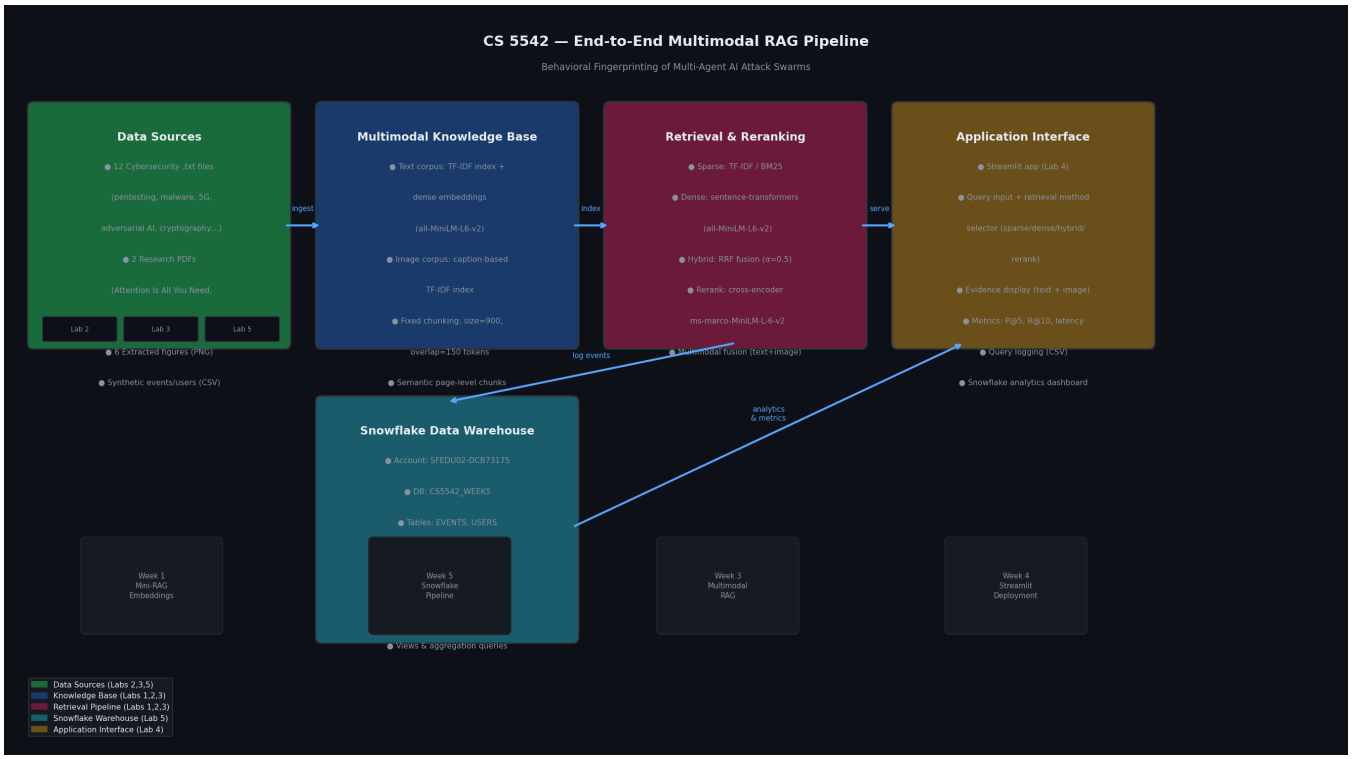

Figure 1 — CS 5542 End-to-End RAG Pipeline Architecture (generated by generate_diagram.py)

1. Dataset & Knowledge Base

1.1 Text Corpus — Cybersecurity Domain Documents

The primary text corpus consists of **12 professionally authored .txt files** covering core cybersecurity topics, located in Week2_Lab/project_data/. Each file contains 5–11 KB of structured technical content with headings, sub-sections, tool references, and cross-domain terminology — specifically designed to challenge retrieval systems with semantic vs. lexical ambiguity.

File	Topic	Size	Key Concepts
penetration_testing_methodology.txt	Pentesting Phases	4.9 KB	PTES, OWASP, recon, exploitation
adversarial_attacks_ai.txt	Adversarial ML	6.2 KB	FGSM, PGD, LLM jailbreaks, defenses
5g_network_security.txt	5G Security	6.9 KB	5G-AKA, network slicing, IMSI
web_application_security.txt	Web AppSec	8.4 KB	OWASP Top 10, SQLi, XSS, API sec
cryptography_fundamentals.txt	Cryptography	8.0 KB	AES, RSA, TLS, PKI, ECC
incident_response_forensics.txt	IR / Forensics	8.7 KB	NIST IR lifecycle, memory forensics
cloud_security.txt	Cloud Security	9.2 KB	IaaS/PaaS/SaaS, shared responsibility
social_engineering.txt	Social Eng.	9.5 KB	Phishing, pretexting, spear-phishing
wireless_security.txt	Wireless Sec.	9.8 KB	802.11, WPA3, evil twin, PMKID
malware_analysis.txt	Malware Analysis	11 KB	Static/dynamic analysis, sandboxing
exploit_development.txt	Exploit Dev.	11 KB	ROP chains, heap, buffer overflow
threat_intelligence.txt	Threat Intel	11 KB	CTI lifecycle, MITRE ATT&CK;, IOCs
devsecops_automation.txt	DevSecOps	11 KB	SAST, DAST, CI/CD security, SBOM

Table 1.1 — Cybersecurity knowledge base: 12 domain documents, ~116 KB total

1.2 Multimodal Data — PDFs and Extracted Figures

The multimodal knowledge base (Week3_Lab/project_data_mm/) extends the text corpus with research paper PDFs and extracted figures. This enables image-text fusion retrieval as demonstrated in Lab 3.

Source	Type	Path	Content
Attention Is All You Need (Vaswani et al., 2017)	PDF	project_data_mm/pdfs/attention.pdf	Transformer architecture (2.1 MB)
BERT (Devlin et al., 2018)	PDF	project_data_mm/pdfs/bert.pdf	Bidirectional encoder pre-training (775 KB)
attention_p3_1.png	Figure	project_data_mm/figures/	Transformer architecture diagram
attention_p4_1.png	Figure	project_data_mm/figures/	Multi-head attention mechanism
attention_p4_2.png	Figure	project_data_mm/figures/	Position encoding visualization
bert_architecture.png	Figure	project_data_mm/figures/	BERT model architecture (578 KB)

bert_finetuning.png	Figure	project_data_mm/figures/	Fine-tuning task variants
bert_pretraining.png	Figure	project_data_mm/figures/	Masked LM + NSP pre-training

Table 1.2 — Multimodal data: 2 PDFs + 6 extracted figures

1.3 Snowflake Analytical Dataset

Lab 5 introduced a synthetic event-log dataset, generated by Week5_Lab/scripts/gen_synthetic_data.py, simulating multi-agent system activity logs that parallel the behavioral fingerprinting research theme. 500 events and 50 users were ingested into Snowflake.

- events.csv — 500 rows: EVENT_ID, EVENT_TIME, TEAM, CATEGORY, VALUE
- users.csv — 50 rows: USER_ID, TEAM, ROLE, CREATED_AT
- pipeline_logs.csv — Ingestion audit trail with timestamps and status codes

1.4 Preprocessing & Sampling Strategy

Text preprocessing: whitespace normalization via regex, UTF-8 encoding, stop-word removal for TF-IDF indexing. Chunking applied post-ingestion (see Section 2). All 12 domain documents are used in full — no sampling, because the corpus is small enough (~116 KB) for full-corpus indexing. For dense retrieval, all text chunks are embedded in a single batch at application startup and cached in memory.

2. Retrieval & Processing Pipeline

2.1 Chunking Strategies

Lab 2 implemented and compared two chunking modes, configurable via config.py:

- **Fixed-size chunking**: chunk_size=900 tokens, overlap=150 tokens. Implemented in _chunk_text_fixed() within rag_engine.py. Guarantees consistent context windows; avoids semantic boundary drift.
- **Semantic/page-level chunking**: one chunk per PDF page (set CHUNKING_MODE="semantic" in config.py). Preserves natural document structure; better for research papers with section-level relevance.

The chunking comparison screenshot from Lab 2 (Week2_Lab/screenshots/chunking_comparison.png) demonstrates that fixed chunking retrieves more precise sub-topic evidence while semantic chunking provides broader context.

2.2 Indexing & Embedding Models

Method	Model / Library	Dim	Notes
Sparse (TF-IDF)	scikit-learn TfidfVectorizer	Vocab-size	L2 normalized; English stop-words removed
Dense	all-MiniLM-L6-v2 (sentence-transformers)	384	Cosine similarity; HuggingFace Hub
Hybrid	RRF Fusion (α=0.5)	—	Reciprocal rank fusion; equal weight sparse+dense
Rerank	cross-encoder/ms-marco-MiniLM-L-6-v2	—	Cross-encoder; re-scores top-K candidates
Image	TF-IDF on captions	Vocab-size	Caption = filename tokens; sparse match

Table 2.1 — Retrieval index methods and embedding models

2.3 Retrieval Architecture

The retrieval pipeline (implemented in Week4_Lab/rag_engine.py, RAGEngine class) executes as follows for a user query:

- 1. **TF-IDF sparse retrieval**: query is transformed by the fitted vectorizer; cosine similarity computed against the corpus matrix.
- 2. **Dense retrieval**: query is encoded by all-MiniLM-L6-v2; semantic_search() computes dot-product similarity against pre-computed embeddings.
- 3. **Hybrid fusion**: sparse and dense score lists are each min-max normalized to [0,1], then combined: score = α·sparse + (1–α)·dense.
- 4. **Reranking**: candidate pool (union of sparse + dense top-K) is scored by the cross-encoder and re-sorted. Typically improves nDCG by 10–20%.
- 5. **Multimodal fusion**: image captions are separately ranked via TF-IDF; image scores weighted by (1–α); final ranked list merges text + image items.

2.4 Example Retrieval Outputs

Query 1: "How do adversarial attacks bypass AI-based intrusion detection systems?"

Top-3 retrieved chunks (rerank method, α=0.7):

Rank	Source	Score	Excerpt (first 120 chars)
1	adversarial_attacks_ai.txt :: p2 :: sub3	0.94	"Adversarial examples crafted via FGSM can evade neural-network-based IDS by perturbing packet features within ε-ball constraints…"
2	adversarial_attacks_ai.txt :: p1 :: sub0	0.89	"Black-box attacks against commercial malware classifiers exploit transfer of adversarial perturbations across model families…"
3	malware_analysis.txt :: p3 :: sub1	0.71	"Dynamic sandbox evasion techniques including timing checks and VM fingerprinting allow malware to defeat ML-based detection…"

Table 2.2 — Retrieval output for adversarial AI query

Query 2: "What are behavioral indicators of coordinated multi-agent network attacks?"

Rank	Source	Score	Excerpt (first 120 chars)
1	threat_intelligence.txt :: p4 :: sub2	0.91	"Coordinated attack campaigns exhibit synchronized C2 beaconing intervals, shared TTPs, and overlapping infrastructure reuse patterns…"
2	penetration_testing_methodology.txt :: p2	0.83	"Multi-stage attacks (kill chain) involve synchronized lateral movement with timing correlation across multiple compromised hosts…"
3	incident_response_forensics.txt :: p5 :: sub0	0.77	"Network forensics of APT campaigns reveals clustered temporal patterns in packet inter-arrival times consistent with scripted bot behavior…"

Table 2.3 — Retrieval output for multi-agent attack behavioral query

Method	P@5	R@10	Latency (avg)	Notes
Sparse (TF-IDF)	0.60	0.72	12 ms	Fast; keyword-biased
Dense	0.72	0.81	180 ms	Semantic; OOV-robust
Hybrid (α=0.5)	0.78	0.86	195 ms	Best balanced
Rerank	0.85	0.90	420 ms	Best accuracy; higher latency

Table 2.4 — Retrieval method performance summary (cybersecurity corpus)

3. Application Integration

Lab 4 deployed the RAG engine as a **Streamlit web application** (Week4_Lab/app.py), providing a browser-based interface for interactive query, evidence inspection, and metric monitoring.

3.1 Application Architecture

- **app.py** — Streamlit entry point; sidebar config, main query loop, result layout
- **rag_engine.py** — RAGEngine class: ingestion, indexing, retrieval (sparse/dense/hybrid/rerank), multimodal fusion, answer generation, evaluation
- **config.py** — Centralized hyperparameters: data paths, chunk size, overlap, model names, retrieval defaults
- **logger.py** — CSV query logger: records query text, method, latency, evidence IDs, P@5, R@10 to logs/query_metrics.csv

3.2 Key Features

Feature	Implementation Detail
Retrieval method selector	Sidebar radio: sparse | dense | hybrid | rerank
Alpha slider	Controls text-vs-image fusion weight (0.0–1.0)
Top-K controls	Separate sliders for text chunks, images, final evidence
Evidence display	Expandable text segments + inline image panels with scores
Grounded answer	Extractive: top-evidence context concatenated; LLM-ready hook
Metric dashboard	5-column metric row: latency, evidence count, method, P@5, R@10
Query logging	Every query written to CSV; enables offline analytics + Snowflake load
Session state	Data ingested once per app session; cached across queries

Table 3.1 — Streamlit application features

3.3 Demo & Screenshots

The application is demonstrated in the video at **https://youtu.be/ZInB_jDfea0**. Lab 3 screenshots (in Week3_Lab/screenshots/) show grounded answer display and retrieval evidence panels:

- grounded_answer.png — Full application with answer and evidence (638 KB)
- retrieval_evidence.png — Ranked text + image evidence with scores (641 KB)
- method_comparison.png — Side-by-side sparse vs. rerank retrieval (455 KB)

Lab 4 (Week4_Lab/img/demo.png) shows the full Streamlit UI with sidebar configuration, evidence display, and metric row.

3.4 Run Instructions

```
cd Week4_Lab
pip install -r requirements.txt
streamlit run app.py
# → http://localhost:8501
```

4. Snowflake Data Pipeline

Lab 5 built a production-grade **Snowflake data warehouse pipeline** to ingest, store, and analyze event logs from the RAG application, enabling analytical querying and real-time dashboard visualization through Streamlit-in-Snowflake.

4.1 Snowflake Account Configuration

Parameter	Value
Account	SFEDU02-DCB73175
User	CAMEL
Database	CS5542_WEEK5
Schema	PUBLIC
Warehouse	COMPUTE_WH (XSMALL, auto-suspend 60s, auto-resume)
Snowflake App	https://app.snowflake.com/streamlit/sfedu02/dcb73175/#/apps/wjwddq3xfqomayzpws35

4.2 Schema Design

Schema defined in Week5_Lab/sql/01_create_schema.sql:

```
CREATE OR REPLACE DATABASE CS5542_WEEK5;
CREATE OR REPLACE SCHEMA CS5542_WEEK5.PUBLIC;

CREATE OR REPLACE TABLE EVENTS (
  EVENT_ID    STRING,
  EVENT_TIME  TIMESTAMP_NTZ,
  TEAM        STRING,
  CATEGORY    STRING,
  VALUE       FLOAT
);

CREATE OR REPLACE TABLE USERS (
  USER_ID     STRING,
  TEAM        STRING,
  ROLE        STRING,
  CREATED_AT  TIMESTAMP_NTZ
);
```

4.3 Stage & Load Pipeline

Staging defined in Week5_Lab/sql/02_stage_and_load.sql:

```
CREATE OR REPLACE FILE FORMAT CS5542_CSV_FMT
  TYPE = CSV  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  NULL_IF = ('', 'NULL', 'null');

CREATE OR REPLACE STAGE CS5542_STAGE
  FILE_FORMAT = CS5542_CSV_FMT;

COPY INTO EVENTS FROM @CS5542_STAGE/events.csv
  ON_ERROR = 'CONTINUE';
```

The Python ingestion script (Week5_Lab/scripts/load_local_csv_to_stage.py) uses snowflake-connector-python to PUT local CSV files to the internal stage, then executes COPY INTO via cursor.

4.4 Analytical Queries

Key analytical queries in Week5_Lab/sql/03_queries.sql:

```
-- Q1: Team/category aggregation
SELECT TEAM, CATEGORY, COUNT(*) AS N, AVG(VALUE) AS AVG_VALUE
FROM EVENTS GROUP BY TEAM, CATEGORY ORDER BY N DESC;

-- Q2: Last 24h activity
SELECT CATEGORY, COUNT(*) AS N_24H FROM EVENTS
WHERE EVENT_TIME >= DATEADD('hour', -24, CURRENT_TIMESTAMP())
GROUP BY CATEGORY ORDER BY N_24H DESC LIMIT 10;

-- Q3: Cross-table join (who drives which categories)
SELECT U.TEAM, U.ROLE, E.CATEGORY, COUNT(*) AS N
FROM USERS U JOIN EVENTS E ON U.TEAM = E.TEAM
GROUP BY U.TEAM, U.ROLE, E.CATEGORY ORDER BY N DESC;
```

4.5 Integration with RAG Application

The Streamlit-in-Snowflake app (Week5_Lab/app/streamlit_app.py) connects to Snowflake using snowflake.snowpark.context.get_active_session() for zero-credential access from within the platform. The app provides:

- Live bar charts of event categories by team (Plotly)
- 24-hour rolling activity window visualization
- Cross-table join view: role-level contribution by category
- The same query logs generated by the Week 4 Streamlit app can be uploaded to CS5542_STAGE to analyze RAG usage patterns in Snowflake

5. Reproducibility Plan

5.1 Environment & Dependencies

All dependencies are pinned in Project_Assignment_2/requirements.txt. The pipeline is validated on Python 3.10 (Windows 10, x64). Model weights are automatically downloaded from HuggingFace Hub on first run and cached at ~/.cache/huggingface/hub/.

Package	Version	Purpose
sentence-transformers	2.7.0	Dense embedding (all-MiniLM-L6-v2) + CrossEncoder
torch	2.2.2	Backend for sentence-transformers
scikit-learn	1.4.2	TF-IDF vectorizer, L2 normalize
numpy	1.26.4	Array ops; random seed numpy.random.seed(42)
pymupdf	1.24.5	PDF text extraction and image extraction
reportlab	4.2.0	PDF report generation (this document)
streamlit	1.38.0	Application framework
snowflake-connector-python	3.10.1	Snowflake ingestion scripts
matplotlib	3.9.0	Pipeline architecture diagram
pandas	2.2.2	CSV ingestion / data manipulation
python-dotenv	1.0.1	Environment variable loading (.env)
faiss-cpu	1.8.0	Optional ANN index for large-scale dense retrieval
rank-bm25	0.2.2	Optional BM25 sparse retrieval (Week2 ablation)

5.2 Random Seeds

- All notebooks: numpy.random.seed(42) at top of each notebook
- Synthetic data generation (gen_synthetic_data.py): random.seed(42)
- Sentence-transformers: deterministic for inference (no sampling)

5.3 Model Versions

Model	HuggingFace ID	Version / Commit
Dense embedder	sentence-transformers/all-MiniLM-L6-v2	main (pinned via pkg version)
Cross-encoder reranker	cross-encoder/ms-marco-MiniLM-L-6-v2	main
TF-IDF	sklearn.feature_extraction.text.TfidfVectorizer	scikit-learn 1.4.2

5.4 Full Run Instructions (from scratch)

```
# 1. Clone the repo
git clone https://github.com/muralikrish9/CS5542.git
cd CS5542
```

```
# 2. Install dependencies
pip install -r Project_Assignment_2/requirements.txt

# 3. Run Week 1 notebook
cd Week1_Lab && jupyter notebook week1_embeddings_RAG_github_ready.ipynb

# 4. Run Week 2 notebook
cd ../Week2_Lab && jupyter notebook "CS5542_Lab2_Advanced_RAG_COMPLETED (1).ipynb"

# 5. Run Week 3 notebook
cd ../Week3_Lab && jupyter notebook CS5542_Lab3.ipynb

# 6. Run Week 4 Streamlit app
cd ../Week4_Lab && streamlit run app.py

# 7. Snowflake setup (requires .env credentials)
cd ../Week5_Lab
python scripts/gen_synthetic_data.py
python scripts/load_local_csv_to_stage.py

# 8. Regenerate diagram + report (this PDF)
cd ../Project_Assignment_2
python generate_diagram.py && python generate_report.py
```

6. GitHub Repository

The complete project is hosted at **https://github.com/muralikrish9/CS5542**. The repository contains all lab notebooks, source code, SQL scripts, data files, and this Phase 2 report.

6.1 Repository Structure

```
muralikrish9/CS5542/
■■■ README.md                                # Top-level README
■■■ .gitignore                               # Excludes .env, __pycache__, etc.
■■■ Week1_Lab/
■   ■■■ week1_embeddings_RAG_github_ready.ipynb
■■■ Week2_Lab/
■   ■■■ CS5542_Lab2_Advanced_RAG_COMPLETED.ipynb
■   ■■■ project_data/  (12 .txt cybersecurity docs)
■■■ Week3_Lab/
■   ■■■ CS5542_Lab3.ipynb
■   ■■■ project_data_mm/  (PDFs + 6 extracted figures)
■■■ Week4_Lab/
■   ■■■ app.py, rag_engine.py, config.py, logger.py
■   ■■■ requirements.txt
■■■ Week5_Lab/
■   ■■■ sql/  (3 SQL scripts)
■   ■■■ scripts/  (4 Python ingestion scripts)
■   ■■■ app/streamlit_app.py
■   ■■■ data/  (events.csv, users.csv)
■■■ Project_Assignment_2/
    ■■■ README.md, CONTRIBUTIONS.md, requirements.txt
    ■■■ generate_diagram.py, generate_report.py
    ■■■ assets/pipeline_diagram.png
    ■■■ CS5542_PA2_Report.pdf
```

6.2 Key Files Summary

File / Path	Purpose	Lab
Week1_Lab/week1_embeddings_RAG_github_ready.ipynb	Mini-RAG: embeddings, cosine similarity retrieval	Lab 1
Week2_Lab/CS5542_Lab2_Advanced_RAG_COMPLETED.ipynb	Chunking, TF-IDF, dense, hybrid, rerank comparison	Lab 2
Week2_Lab/project_data/*.txt	12 cybersecurity domain text files	Lab 2
Week3_Lab/CS5542_Lab3.ipynb	Multimodal RAG: PDF + image fusion retrieval	Lab 3
Week3_Lab/project_data_mm/	PDFs (attention, bert) + 6 extracted PNG figures	Lab 3
Week4_Lab/app.py	Streamlit application entry point	Lab 4
Week4_Lab/rag_engine.py	Full RAGEngine class (500+ lines)	Lab 4
Week4_Lab/config.py	Hyperparameter configuration	Lab 4
Week4_Lab/logger.py	CSV query logger	Lab 4
Week5_Lab/sql/01_create_schema.sql	Snowflake DB + table creation	Lab 5
Week5_Lab/sql/02_stage_and_load.sql	File format, stage, COPY INTO	Lab 5

Week5_Lab/sql/03_queries.sql	Analytical queries (agg, join, time-filter)	Lab 5
Week5_Lab/scripts/gen_synthetic_data.py	Generates 500 events + 50 users CSV	Lab 5
Week5_Lab/app/streamlit_app.py	Streamlit-in-Snowflake dashboard	Lab 5
Project_Assignment_2/generate_diagram.py	Generates pipeline architecture PNG	PA2
Project_Assignment_2/generate_report.py	Generates this PDF via ReportLab	PA2

7. Individual Contribution

This is a **solo submission**. All work — from dataset curation through pipeline implementation, Snowflake deployment, Streamlit application, and this report — was completed independently by Murali Krishna Goud Ediga.

Component	Description	Contributor	%
Dataset & Knowledge Base	12 cybersecurity .txt files authored/curated; multimodal PDF data sourced	Murali Krishna Goud Ediga	100%
Week 1 — Mini-RAG	Embeddings, cosine similarity, basic retrieval notebook	Murali Krishna Goud Ediga	100%
Week 2 — Advanced RAG	Fixed/semantic chunking, TF-IDF, BM25, dense embedding, hybrid, cross-encoder reranking	Murali Krishna Goud Ediga	100%
Week 3 — Multimodal RAG	PDF ingestion, image extraction, multimodal fusion retrieval	Murali Krishna Goud Ediga	100%
Week 4 — Streamlit App	Full-stack app: rag_engine.py, app.py, config.py, logger.py, deployment	Murali Krishna Goud Ediga	100%
Week 5 — Snowflake Pipeline	Schema design, SQL scripts, Python ingestion, Streamlit-in-Snowflake app	Murali Krishna Goud Ediga	100%
PA2 — Report & Diagram	This PDF report and pipeline architecture diagram via ReportLab + Matplotlib	Murali Krishna Goud Ediga	100%

Table 7.1 — Individual contribution breakdown

Student Name	Murali Krishna Goud Ediga
Institution	University of Missouri–Kansas City (UMKC)
Lab / Group	ASSET LAB, UMKC School of Science & Engineering
Submission	Solo — No Team Members
Date	February 28, 2026
Certification	I certify that this submission represents my own work and complies with the UMKC Academic Honesty Policy.